

LOCAL AI · YOUR HARDWARE · YOUR DATA

4 000 000 · 11 · 1

FILES

KNOBS

ANSWER

LLAMATOASTER

the appliance that finds it.

Running a model locally shouldn't require becoming a AI expert.

4 million files — no one labeled for you

200000

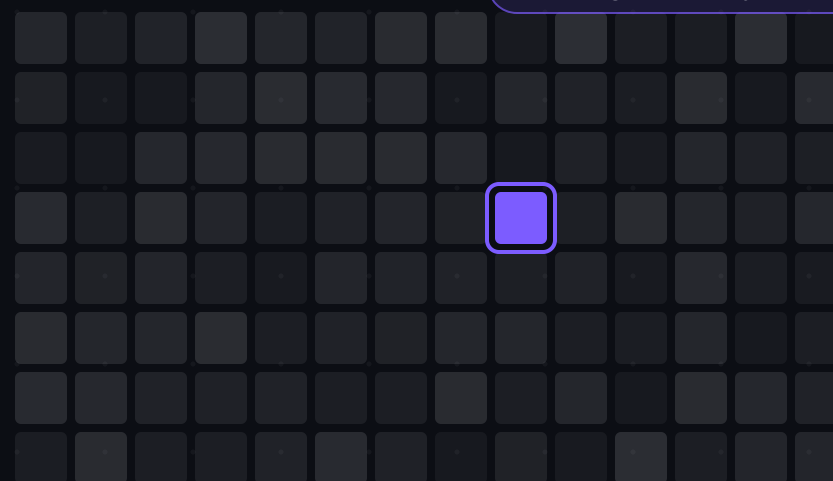
+ GGUF repositories on Huggingface

4 000 000

files once every quantization is counted

DeepSeek · Qwen · Llama · Gemma · Kimi · GLM · Mistral · Nemotron

◆ the right one for your box



The index grows weekly. Last month's "best" is already stale.
So everyone picks by folklore: a Reddit thread said that one is good.

Choosing is one haystack. Configuring it is another.

11 knobs, cross-multiplied
on every run, on every machine, for every model



The first config that doesn't crash wins — not the best one.
Complexity isn't overcome; it filters people out.

- 01 One bad knob — silent quality loss, or a hard crash**
Wrong KV cache → degraded tokens. Context too large → OOM at hour three.
- 02 One good config on GPU X — broken on GPU Y**
Copied recipes fail across architectures, drivers, OS versions.
- 03 The stale oracle**
Ask a chatbot for the best local setup and you get confident folklore from a training set that closed before the model you want existed.

Quality is measured to death. Performance is a black box.

- Dozens of leaderboards rank "smartness" — every release scored within days.
OpenLLM · MMLU-Pro · IFEval · BBH · MuSR · GPQA · HELM · BIG-Bench · ARC.
- Almost nothing answers the other half:
processing speed · generation speed · TTFT · usable context — on YOUR hardware.

Mountains of quality scores. Performance is still trial and error.
The hardware was almost always fine. The choice layer is what fails.

QUALITY — SCORED NIGHTLY

every release



answers for
your hardware

processing speed

generation speed

TTFT

usable context

Models crossed the line — but accessible ≠ obvious

Proprietary APIs bring outages, refusals, price changes — none of it yours.
Your data stays on your machine, off every provider's logs.

- ✗ **Chasing the biggest model**
→ swap, crash, 0.3 tok/s
- ✗ **Chasing hype**
→ the loud model instead of the right one for YOUR task
- ✗ **Following last month's advice**
→ already stale

THE LINE MODELS JUST CROSSED

**A 27B model on one consumer GPU now
lands where
last generation's flagship did.**

Every month brings another "impossible" small model.

**The missing piece is maximum effect for your workload — not
maximum parameters, not the loudest release.**

no outages

no refusals

no price changes

✦ nothing leaves your box

An orchestrator that answers, not just measures

One command connects any GPU or CPU box. Pull-only: the worker long-polls out, the server never dials in. Nothing to port-forward, nothing to firewall-open.

You state intent — goal × workload × target context.

The grid builds itself. The sweep runs itself, in three linked stages:

tuning → refine → sweep.

Intent in.

Decision out.

Set up tonight. By morning: "run this exact configuration" — not "go figure it out."



Out come four profile cards — Max Speed · Balanced · Max Context · Low Memory — each with the exact command line and a human-readable reason it won.

```
llama-bench -m model.gguf -fa on -ctk q8_0 -ctv q8_0 -ngl 31 -b 512 -ub 512 -d 4096
```

— ALREADY BUILT

Not a roadmap — this is what runs today



Models

search Hugging Face, download a quant straight onto a chosen machine



Builds

install, activate and delete llama.cpp releases from the web UI, no compiler



Load test

6 modes bisect the real context ceiling instead of guessing it



Curves

tok/s and TTFT against growing context; the concurrency knee, derived on read



Speculative decoding

MTP runs benchmarked through llama-server, not just llama-bench



Quality

llama-perplexity PPL and KLD against a pinned corpus, so degradation is measured



Decentralized network

many machines, one queue; device-flow enrolment, per-user isolation in SQL



Assistant

your own hardware, models and results as context, plus opt-in community aggregates



Exchange

JSON · CSV · Markdown export, and importable bundles that keep their methods

Nine surfaces, one queue, **one command to attach a machine**.
Every one of these is in the repository today — none of it is a promise.

RUNS TODAY

Every number carries its methods

- ✓ Four eligibility gates — stability, suspect samples, missing pairs, caveat flags — send timer-bug results like 1e6 tok/s to the tally, not into the average. Every rejection is counted and shown: a silent zero-card outcome would be indistinguishable from a bug.
- ✓ Memory is estimated per-tensor from the real GGUF, not from file size ÷ layers — it catches the config that logs "31/31 layers offloaded" while quietly paging into system RAM.
- ✓ No cell renders a number without naming its source: measured here, derived from llama-bench, or unavailable. Method versions are never averaged across.
- ✓ Comparisons are fair by construction — a different machine, build or GPU rejects the member, not the verdict. Exports carry a hash per row, and community aggregates are k-anonymised in SQL: never a group under 5 contributors, opt-in, never your own rows.

Max Speedmethod v3 · 5 repeats

tg q8_0/q8_0 ngl 31	42.7 tok/s
pp q8_0/q8_0 ngl 31	918 tok/s
tg f16/f16 ngl 33	1.0e6 tok/s

rejected · suspect samples

config_hash a91f2c7e... · ctx 16 384 verified

VERIFIED
PROBED

— WHERE IT SITS

Everyone else solves "it runs." We solve "run it this way."

TOOL	WHAT IT GIVES YOU	WHAT IT NEVER ANSWERS
Ask a chatbot	An instant confident answer	Whether it was ever true
Leaderboards	Which model is smartest	What will it do on MY GPU?
Ollama · LM Studio	One-click run, sane defaults	Is this the best config for THIS box?
llama-bench	Honest numbers for one command line	Which of 4 000 combinations should I run?
LlamaToaster	A ranked decision, with the command line	Output quality — until you run a quality pass

The gap isn't running models. **It's choosing how.**

Run your rig tonight

1

command connects your rig

a spare GPU box, an old laptop, or the CPU on your VPS

llamatoaster.com

01

Connect your machine

One command. Pull-only — nothing to port-forward, no manual llama.cpp build.

02

Find your optimal config

State intent — goal × workload × target context — and get the profile that wins on your box.

03

Create a global local inference database together

Opt in — every validated result makes the catalog searchable for everyone.